AutoVet

Capstone Defense Guide

Simple, basic answers to every requirement in Part 2 (System Readiness), Part 3 (Question-and-Answer Session) and Part 5 (Evaluation and Panel Decision).

How to use this guide	
Blue boxes	The panel's requirement, copied word for word.
Text under it	Our answer: what AutoVet has, how to show it, and what to say.
[Check manuscript]	Something only your paper can answer (for example, your exact SDLC model or your cited studies). Fill it in from your manuscript; do not guess.
Yellow boxes	Something to do before the defense, or a limitation to state honestly.

Facts in this guide were checked against the AutoVet source code and project documents on November 7, 2026. If the system changes after that date, re-check the numbers.

Contents

- PART 2. System Readiness (4 requirements)
- PART 3. Question-and-Answer Session (6 topics + source code walkthrough)
- PART 5. Evaluation and Panel Decision (7 criteria)
- Appendix A. Recent improvements (November 7, 2026)
- Appendix B. Known limitations (state these honestly)

Why the earlier PDF was called "Security and Audit Log Update": it only described the changes made on November 7 (sign-up protection and audit logs). It did not answer the defense requirements one by one. This guide replaces it and answers every requirement. Those changes are now in Appendix A.

PART 2. SYSTEM READINESS

2.1 Fully operational and ready for demonstration

> ***Requirement:*** *"The system must be fully operational and ready for demonstration."*

AutoVet is deployed online and works as one system made of these parts:

Part	Where it runs	Address	Status
Admin app (clinic staff)	Vercel	autovet-admin.vercel.app	Online (checked Nov 7, 2026)
Client portal (pet owners)	Vercel	petwellness-web.vercel.app	Confirm before the defense
Backend API + AI forecast	Render (Docker)	autovet.onrender.com	Online (checked Nov 7, 2026)
Database	TiDB Cloud (MySQL-compatible)	-	In use by the backend
Email	Brevo	-	Sends verification, reset and reminder emails
Keep-awake pinger	UptimeRobot	pings /up	Keeps the free Render server from sleeping

Suggested demonstration flow (one account per role):

Step	Who	What to show
1	Pet owner (portal)	Sign up, verify email, add a pet, book an appointment (status: Pending).
2	Staff / Clinic Admin	See the pending booking, approve it, register a walk-in.
3	Veterinarian	Open the patient, add a medical record and vaccination.
4	Clinic Admin	Create and finalize an invoice; stock is deducted automatically (FIFO).
5	Clinic Admin	Open Inventory: low-stock alerts and the AI forecast (days until stock-out).
6	Clinic Admin	Reports and dashboard charts; download a PDF.
7	Clinic Admin	Settings: Audit Logs (who did what), Backups, user roles.
8	Super Admin	Manage clinics, announcements and system-wide logs.

2.2 Testing software, hardware, databases, servers and internet

> ***Requirement:*** *"All necessary software, hardware, databases, servers, and internet connections must be tested."*

Item	How to test it	Passed when
Software - admin app and portal	Sign in with each role and run the demo flow above in Chrome (and one other browser).	Every step works without errors.
Software - automated tests	Run `php artisan test` in the backend folder (MySQL must be running).	All tests pass. On Nov 7, 2026: 83 passed, plus 1 new test added afterwards (passed).
Software - automatic checks	GitHub Actions runs tests, code style, security audit and builds on every push.	Latest run on GitHub is green.
Hardware	Laptop charged + charger, HDMI/adapter to the projector, mouse, phone for the hotspot.	Screen shows on the projector; laptop runs on battery for 1 hour.
Database (online)	Sign in and open lists (patients, inventory).	Data loads in a few seconds.

Item	How to test it	Passed when
Database (local backup copy)	Start MySQL in Laragon and open the local system.	Local system shows the demo data.
Servers	Open autovet.onrender.com/up and autovet-admin.vercel.app.	Both open (the API answers with status 200).
Internet	Test the venue Wi-Fi and the mobile hotspot; open the live system on both.	Pages load on both connections.
Email	Sign up with a test email or use Forgot Password.	The email arrives.

2.3 Backup copies

Requirement: *"Prepare backup copies of the system, database, source code, and other necessary files."*

Backup of	How to make it	Where to keep it
Source code	All code is on GitHub (github.com/andreidcnyy/AutoVet, branch Seb1). Also download it as a ZIP.	GitHub + USB drive + cloud drive
Database	In the admin app: **Settings > Backup & Restore > Create backup**, then **Download**. Or run `php artisan db:backup`. The backup is a compressed file of CSV tables with the key clinic data.	USB drive + cloud drive
Whole system (working copy)	A laptop with Laragon, the code, the local MySQL database and the Python forecast libraries installed, tested to run offline.	Demo laptop (and a second laptop if possible)
Other files	Manuscript, slides, these guides, user manual, UAT forms, the .env settings file (contains passwords: keep it private, never on GitHub).	USB drive + cloud drive + printed copies

Important: Download the database backup right after creating it. Render's free server wipes its own disk on every redeploy, so a backup left only on the server can disappear. Also note: the backup can be downloaded and read, but restoring a CSV backup inside the app is not available.

2.4 Contingency plans

Requirement: *"Prepare contingency plans in case of technical difficulties."*

If this happens	Do this
The live site is slow on the first load	The free Render server is waking up. Wait about one minute and reload. Open the site 1 hour early to avoid this.
No internet at the venue	Switch to the mobile hotspot.
Render, Vercel or TiDB is down	Run the demonstration on the local copy (Laragon) on the laptop.
The laptop fails	Use the second laptop or a groupmate's laptop with the local copy, or open the live site from any browser.
Projector does not work	Present from the laptop screen; have printed screenshots of key screens.
"Too many attempts" message	The rate limit is working (you can explain this as a security feature). Wait a minute or use another prepared account.
Verification email does not arrive	Check spam; use an already verified demo account.
A feature fails during the demo	Stay calm, explain what it should do, show the screenshot or the source code, and continue.

PART 3. QUESTION-AND-ANSWER SESSION

The Q&A lasts up to **45 minutes**, with a possible **10-minute extension**. **All group members** must be ready to answer questions about the whole project, and may be asked to **show and explain specific source code**. Divide the topics below, but every member should read all of them.

3.1 Chapters 1-3

Requirement: *"Chapters 1-3: Background, objectives, problem statement, related literature, methodology, and system design."*

Q: Background - What is AutoVet?

A: AutoVet is a veterinary clinic management system. Pet owners book appointments and manage their pets online. Clinic staff manage patients, walk-ins, schedules, medical records, billing and inventory from one dashboard. It also forecasts when stock will run low using linear regression. It was built for Pet Wellness Animal Clinic.

Q: Problem statement - What problem does it solve?

A: Clinics that rely on paper logbooks and spreadsheets have slow booking, records that are hard to find, manual billing, and stock that runs out without warning. AutoVet puts these in one digital system. [Check manuscript: use the exact wording of your problem statement.]

Q: Objectives - What are your objectives?

A: (1) Automate appointment booking, scheduling and clinic transactions. (2) Centralize pet profiles, medical histories and vaccination records. (3) Prevent stock-outs with FIFO batch tracking and AI demand forecasting. (4) Generate invoices, receipts and reports. (5) Secure patient and owner data with role-based access and audit logging. (6) Support clinic operations when the connection is unstable. [Check manuscript: match your exact general and specific objectives.]

Q: Related literature - What studies support your work?

A: [Check manuscript: name 2-3 of your cited studies and say how AutoVet is similar and different.] Sources already used in our documents: Hyndman & Athanasopoulos (2021) on forecasting and train/test evaluation; James et al. (2021) on statistical learning; ISO 9241-11:2018 for usability (effectiveness, efficiency, satisfaction); Chai & Draxler (2014) on MAE and RMSE.

Q: Methodology - How did you develop the system?

A: [Check manuscript: state your software development model, for example Agile or Waterfall, and its phases.] Testing methods we used: unit and integration testing with PHPUnit, black-box functional (system) testing, and user acceptance testing with ISO 9241-11 questionnaires on a 4-point Likert scale, summarized with the weighted mean.

Q: System design - How is the system designed?

A: Three layers: (1) two web front-ends built with React (admin app and client portal), (2) a Laravel REST API backend that holds all business rules and security, and (3) a MySQL-compatible database (TiDB). A Python module does the forecasting. The front-ends only talk to the backend through the API (about 190 endpoints). [Check manuscript: refer to your context diagram, ERD and use-case diagram.]

3.2 Chapter 4

Requirement: *"Chapter 4: System development, implementation, testing, evaluation, results, and discussion."*

Q: Development - What tools did you use?

A: Backend: Laravel 12 (PHP 8.2). Front-ends: React 19 with Vite and Tailwind CSS (admin in JavaScript, portal in TypeScript). Database: MySQL 8.4 locally, TiDB online. Forecasting: Python with scikit-learn, pandas and NumPy. Version control: Git and GitHub.

Q: Implementation - How was it deployed?

A: Front-ends on Vercel, backend on Render as a Docker image (with Python included), database on TiDB Cloud, email through Brevo. Every push to GitHub redeploys the system automatically.

Q: Testing - How did you test it?

A: Unit tests check the forecast maths. Integration (feature) tests send real requests to the API and check the results in a separate test database: roles, login security, clinic separation, appointments, invoices, backups, rate limits, audit logs. On November 7, 2026, 83 tests passed, plus 1 added afterwards. GitHub Actions runs all tests on every push.

Q: Evaluation - How did you evaluate the AI forecast?

A: With an 80/20 holdout test: the model learns from the oldest 80% of each item's days and is tested on the newest 20%. Errors are measured with MAE and RMSE, and compared with three simple methods (average, naive, 7-day moving average).

Q: Results - What were the forecast results?

A: On 24 items: median MAE 2.50 units, median RMSE 2.78 units, median test R^2 0.48. The model beat the naive method on 20 of 24 items and the 7-day moving average on 14 of 24. The simple average was slightly better overall (MAE 2.21) because most usage was steady. (Source: benchmark run on October 25, 2026.)

Q: Results - What did users say (UAT)?

A: [Check manuscript: give your UAT weighted means for each group. If the survey results are not yet collected, say so honestly.] The questionnaires cover effectiveness, efficiency and satisfaction for three groups: veterinarians/administrators, front desk staff, and pet owners.

Q: Discussion - Why didn't the AI beat every method?

A: No method wins on every item. When usage is steady, the long-run average is already a good guess. Our model is close to it, clearly beats the other two methods, and follows changes in usage better. We report the comparison honestly.

3.3 Chapter 5

> ***Requirement:*** *"Chapter 5: Summary, conclusions, achievement of objectives, and recommendations."*

Q: Summary and conclusion - What did you conclude?

A: AutoVet works as a complete clinic management system: online booking, records, billing, inventory and reports in one place, protected by roles and audit logs. The forecast gives useful stock-out warnings and is clearly better than guessing from recent days.

Q: Achievement of objectives - Did you meet all objectives?

A: Mostly yes. Booking, records, invoicing, inventory with FIFO and forecasting, reports, roles and audit logging are working. Be honest about the gap: the offline sync to the cloud exists in the code but is **switched off** in the current online deployment, so the live system needs internet. See the table below.

Q: Recommendations - What should future developers do?

A: Turn on and finish the offline sync; collect more real usage data and re-run the forecast benchmark; try seasonal forecasting methods; add online payment and SMS; add two-factor login; encrypt personal fields in the database; build a mobile app.

Objective	Achieved?	Evidence to show
Automate booking, scheduling, transactions	Yes	Portal booking, approval, walk-ins, double-booking prevention
Centralize pet records and medical history	Yes	Patient profile, medical records, vaccinations
Prevent stock-outs (FIFO + AI forecast)	Yes	Inventory batches, auto-deduction, forecast page
Invoices, receipts, reports	Yes	Invoice PDF, emailed receipts, reports page
Secure data with roles and audit logs	Yes	Role restrictions, Settings > Audit Logs
Work offline and sync to the cloud	Partly	Sync code exists but is off in the online deployment

3.4 Technical Implementation

> ***Requirement:*** *"Technical Implementation: Programming languages, frameworks, database, system architecture, source code, and integrations."*

Topic	Answer
Programming languages	PHP 8.2 (backend), JavaScript (admin app), TypeScript (portal), Python 3 (forecast), SQL (database).
Frameworks	Laravel 12 (backend), React 19 + Vite + Tailwind CSS (front-ends), scikit-learn + pandas (forecast).
Database	MySQL-compatible. TiDB Cloud online, MySQL 8.4 (Laragon) locally. Accessed with Laravel's Eloquent ORM. Foreign keys protect links between tables.
System architecture	Client-server with a REST API. Browser apps call the Laravel API with a login token; the API checks the role, applies the rules, and reads or writes the database. Laravel runs the Python forecast as a separate process.
Source code	One GitHub repository: backend/ (Laravel + ai/ Python), frontend/admin, frontend/portal. See 3.7 for which file to open for each feature.
Integrations	Brevo (email), Google sign-in (portal), Pusher with Laravel Echo (live updates), Cloudflare Turnstile (optional sign-up robot check), UptimeRobot (keep-awake), GitHub Actions (automatic tests), Vercel and Render (hosting).

Q: Why Laravel and React?

A: Laravel gives ready, secure tools for login, validation, database access and email, so we could build faster and more safely. React makes fast, interactive screens. Both are widely used and well documented.

Q: Why linear regression?

A: It is simple, fast, easy to explain to clinic staff, and works with the small amount of data a clinic has. The slope of the line is the daily usage rate. (See the separate PDF: Forecast Linear Regression Equation.)

Q: Why TiDB and Render?

A: Both have free plans. TiDB works like MySQL, which we use locally, so no code changes were needed.

3.5 Security and Data Management

> ***Requirement:*** *"Security and Data Management: Authentication, authorization, data privacy, database security, and backup mechanisms."*

Topic	How AutoVet does it
Authentication (who are you?)	Email + password, checked against a bcrypt hash. Laravel Sanctum then gives the user a login token (expires after 7 days by default) that the app sends with every request. Pet owners must verify their email; Google sign-in is also available. Staff and pet owners are in separate account tables. Rate limits slow down password guessing and fake sign-ups. A password reset signs out all other devices.
Authorization (what may you do?)	Role-based access control with 5 roles: Super Admin, Clinic Admin, Veterinarian, Staff, Pet Owner. Every API route checks the login (auth:sanctum) and the role (role middleware). Hiding a page in the app is only for convenience; the real check is on the server.
Data privacy	Each clinic only sees its own data. Pet owners only see their own pets, appointments and invoices. Error messages never show internal details. The audit log hides passwords and tokens. Privacy Policy and Terms pages are in the portal.
Database security	The database is not open to the public; only the backend connects to it, over an encrypted (TLS) connection. Queries go through Eloquent with bound parameters (protects against SQL injection). Validation runs in the browser, on the server and in the database (foreign keys). Deleted records are archived first and can be restored. The audit log records who changed what.
Backup mechanisms	Settings > Backup & Restore creates a downloadable backup of key clinic data (CSV files in a compressed file); each clinic can only download its own. The same is available as php artisan db:backup. Source code is backed up on GitHub. Limitation: restoring a CSV backup inside the app is not available, and scheduled backups do not run on the free Render plan, so backups are made manually.

Q: How do you stop fake accounts?

A: An account is created only after the email link is clicked. Sign-up is limited to 5 tries per minute and 20 per day per visitor, and 5 per hour per email address. An optional robot check (Cloudflare Turnstile) can be turned on.

Q: Where is the login token stored, and is that safe?

A: In the browser's local storage. If an attacker could run JavaScript on the page (XSS) they could read it. The Content-Security-Policy header reduces this risk. A future improvement is httpOnly cookies.

Q: Is personal data encrypted?

A: Passwords are hashed and all traffic uses HTTPS. The hosting providers encrypt the stored database and files. Individual fields such as phone numbers are not separately encrypted; this is a limitation.

3.6 System Functionality

Requirement: *"System Functionality: Features, workflows, user roles, limitations, and technical decisions."*

Topic	Answer
Features	Online booking with double-booking prevention; walk-ins; vet schedules; patient profiles, medical records and vaccinations; invoicing with PDF and emailed receipts; inventory with FIFO batches, expiry and low-stock alerts; AI stock-out forecast; reports and dashboard; audit logs; archive and restore; backups; rule-based FAQ chatbot; notifications; multi-clinic management.
Workflow: booking	Pet owner books in the portal -> status Pending -> staff approve or decline -> owner is notified -> visit -> medical record -> invoice.
Workflow: billing to inventory	Staff create an invoice with services and products -> finalize -> products are deducted from stock, oldest batch first (FIFO) -> usage history feeds the forecast.
User roles	Super Admin (all clinics), Clinic Admin (runs the clinic), Veterinarian (clinical work), Staff (front desk, no financial reports), Pet Owner (portal only).
Limitations	See Appendix B.

Topic	Answer
Technical decisions	Separate front-ends and API (clear security boundary); linear regression (simple and explainable); soft delete with archive (mistakes can be undone); free cloud services (no cost for the clinic during the study).

3.7 Explaining source code

> ***Requirement:*** *"The panel may also request the proponents to present and explain specific portions of their source code to verify their understanding of the system's technical implementation."*

Open the file, point to the function, and explain it in 2-3 sentences: **what comes in, what it checks, what it saves or returns**.

If asked about	Open this file	Explain
All API routes and who may use them	`backend/routes/api.php`	Groups of routes behind auth:sanctum and role:... checks.
Login	`backend/app/Http/Controllers/AuthController.php (login)`	Finds the account, checks the password hash, blocks inactive accounts, creates a Sanctum token.
Role checking	`backend/app/Http/Middleware/RoleMiddleware.php` `backend/app/Enums/Roles.php`	Returns 403 if the user's role is not allowed.
Double-booking prevention	`backend/app/Http/Controllers/AppointmentController.php (store)`	Checks the pet's daily limit and the vet's time slot before saving.
Invoice and FIFO stock deduction	`backend/app/Services/InvoiceFinalizationService.php (finalizeInvoice)`	Deducts products from the oldest batch first and logs the deduction.
AI forecast	`backend/ai/forecast.py`	Builds the running total of usage, fits LinearRegression, divides remaining stock by the slope.
Audit log	`backend/app/Support/AuditLogger.php` `backend/app/Traits/HasAuditTrail.php`	Model events call record(), which hides secrets and saves who, what, when.
Rate limits	`backend/app/Providers/AppServiceProvider.php`	RateLimiter::for(...) rules for login, sign-up and the API.
Security headers	`backend/app/Http/Middleware/SecurityHeadersMiddleware.php`	Adds HSTS, CSP, X-Frame-Options.
Front-end login	`frontend/admin/src/context/AuthContext.jsx` `frontend/admin/src/api.js`	Saves the token and sends it with each request.
Tests	`backend/tests/Feature`, `backend/tests/Unit`	Each test sends a request and checks the result.

PART 5. EVALUATION AND PANEL DECISION

> ***Requirement:*** *"At the conclusion of the presentation and Q&A, the panel shall evaluate the capstone project based on the following criteria."*

For each criterion: what the panel looks for, what AutoVet has, and what to show.

Criterion	What AutoVet has	Show / say
System functionality and completeness.	Booking, records, billing, inventory with FIFO, AI forecast, reports, roles, audit logs, backups, portal for pet owners. Offline sync is only partly done.	Run the demo flow in 2.1. Use the objectives table in 3.3.
System usability and user experience.	Dashboard design, clear menus, form checks before saving, friendly error messages, works in mobile browsers. UAT based on ISO 9241-11.	Show a short task (booking in under a minute). Give UAT results [Check manuscript].
Technical implementation and performance.	Laravel API + React apps + TiDB + Python forecast; paging and indexes for fast lists; automatic deployment; automated tests on every push.	Explain the architecture (3.4). Show GitHub Actions and the test results.
Security and data management.	Hashed passwords, tokens, 5 roles checked on the server, clinic separation, rate limits, security headers, audit log without secrets, archive/restore, backups.	Use the table in 3.5. Show Settings > Audit Logs.
Compliance with the stated project requirements.	Core requirements met. Not yet met as first written: working offline sync, field encryption, account lockout, auto-logout on inactivity, SMS.	Be honest: list what is done and explain each gap as a limitation and recommendation.
Quality and completeness of documentation.	User manual, system architecture, security, testing summary, forecast benchmark (Chapters 4-5 guide), UAT forms, this guide, comments in the code.	Bring printed and digital copies.
Proponents' knowledge and understanding of the project.	Every member can explain the whole system and the code.	Practise Part 3, especially 3.7, with every member.

Appendix A. Recent improvements (November 7, 2026)

Change	In simple terms
Sign-up protection	Rate limits now apply to each visitor instead of one shared limit for everyone. Sign-up: 5 per minute and 20 per day per visitor, 5 per hour per email. Optional Cloudflare Turnstile robot check (off until its two keys are set).
Audit log upgrade	Passwords and tokens are no longer stored in the log (old entries were cleaned). Each entry saves who did it, their role, the record's name, the real IP address, and only the fields that changed. Sign-ins, failed sign-ins, archives and restores are now recorded. Search and filters were added. Entries older than 365 days are removed automatically so the log does not grow too large.
Role labels	The audit log now shows each person's real role (Clinic Admin, Veterinarian, Staff, Super Admin) instead of labelling every clinic account "Staff".

Appendix B. Known limitations (state these honestly)

- Offline sync to the cloud exists in the code but is switched off; the live system needs internet.
- No online payments; payment is taken at the clinic.
- No SMS; notifications are by email and in the app.

- No two-factor login, no account lockout and no auto-logout after inactivity.
- Personal fields (such as phone numbers) are not separately encrypted in the database.
- The forecast uses only past usage; it does not consider seasons, outbreaks or supplier delays.
- The chatbot is rule-based (fixed FAQ topics).
- Restoring a CSV backup inside the app is not available; scheduled backups do not run on the free Render plan.
- No installable mobile app (works in mobile browsers).